RHEMLO

From demo to live server

A step-by-step deployment guide

Vercel · Google · AWS · Supabase

Version 1 · November 2026

Swipe to turn the page

START HERE

Contents

How to use this guide · 3
What Rhemlo is today · 4
The pieces of a live Rhemlo · 5
Choose your path · 6
Accounts and tools checklist · 7
Put the demo online (Vercel) · 8
Check the public demo · 9
What must be built before “live” · 10
What the migrations already give you · 11
Create the Supabase projects · 12
Run the migrations · 13
Load the manual content · 14
Build the missing pieces · 15
Sign-in settings · 16
Switch the app to live · 17
Staging test checklist · 18
Host on Firebase Hosting · 19
Build and deploy on Firebase · 20
Host on AWS Amplify · 21
AWS rewrites and domain · 22
Your domain and HTTPS · 23
Email that arrives · 24
Security and privacy checklist · 25
Monitoring · 26
Rough monthly costs · 27
Go-live checklist · 28
Rolling back · 29
The journey from here · 30
Glossary · 31

HOW TO READ THIS

How to use this guide

This guide takes Rhemlo from the demo it is today to a live service that real learners sign in to. Read it once end to end, then work through it page by page.

- **Swipe or tap** to move between pages. Each page is one idea or one set of steps.
- **Contents** (page 2) and your PDF app's bookmarks jump straight to any section. The footer link on every page returns to Contents.
- **Grey boxes** are commands or settings to copy exactly.
- **Coloured boxes**: purple is a tip, orange is a caution, red is a must-do before real users, green is the finish line of a phase.

> Prices, plan limits and console screens change often. Where this guide gives a price or a menu name, check it on the provider's website on the day you do it.

The three phases

Phase	What you get	Effort
0 · Public demo	Rhemlo on a real web address, still on the demo server	An afternoon
1 · Live backend	Real accounts, sync across devices, real leagues and community	The real work: weeks
2 · Go live	Domain, email, security, monitoring, launch day	A few days

WHERE YOU ARE

What Rhemlo is today

The app is a **static web app** (React + Vite). Building it produces a folder, `dist`, of plain files: pages, scripts, images and the manual data. Any static host can serve it.

Already built

- All study features, the CBT mock, planner, insights, community, admin and steward screens.
- The manuals as files: `public/data` (about 32 MB) and artwork in `public/assets` (about 76 MB).
- A service worker for offline use (on the published site only).
- **A demo server** that runs inside each browser, with demo accounts and simulated people.
- **A live adapter** for Supabase, switched on by three settings, with no code change to the screens.

> The demo server keeps everything in the visitor's own browser. Two phones never see each other's data. That is why a real backend (Phase 1) is needed before launch.

THE BIG PICTURE

The pieces of a live Rhemlo

Piece	Job	Recommended
Web host + CDN	Serves the app files fast, worldwide, over HTTPS	Vercel (Google and AWS options later)
Database + API	Accounts, progress, leagues, chat, moderation	Supabase (managed Postgres)
Sign-in	Passwords, email links, sessions	Supabase Auth
Scheduled jobs	Weekly league results, clean-ups	Postgres pg_cron inside Supabase
Email	Sign-in links, consent requests, reports	Resend or Postmark via SMTP
Domain	Your own address, e.g. rhemlo.app	Any registrar
Monitoring	Errors and uptime	Sentry + an uptime checker

The browser talks to Supabase directly with a public “anon” key. Security comes from **row-level security** rules inside the database, which the bundle’s migrations already define.

DECISION

Choose your path

The app's live adapter speaks Supabase's API (`/auth/v1` for sign-in and `/rest/v1/rpc` for database functions). So keep **Supabase as the backend on every path**. Only the web host changes.

Path	Web host	Backend	Choose it if…
A · best fit	Vercel	Supabase Cloud	You want the fastest, simplest route. The repo already has vercel.json.
B · Google	Firebase Hosting	Supabase Cloud	Your organisation standardises on Google Cloud.
C · AWS	Amplify Hosting	Supabase Cloud	Your organisation standardises on AWS.

Moving the database itself to Google Cloud SQL or AWS RDS means replacing Supabase Auth and its API layer: a large rewrite. Supabase can be self-hosted on a Google or AWS server later if data must live in your own cloud account.

Pick a Supabase region close to your learners (for Nigeria, a European region such as London is usually closest). Region cannot be changed later without moving the project.

BEFORE YOU START

Accounts and tools checklist

Accounts (free to open)

- **GitHub**: the code is at github.com/tiwaomotosho/rhemlo_app.
- **Vercel**: sign up with GitHub. Or Firebase / AWS for paths B and C.
- **Supabase**: sign up with GitHub.
- **Domain registrar**: Namecheap, Cloudflare, or the one your church already uses.
- **Email sender**: Resend or Postmark.
- **Sentry** (optional): error reports.

Tools on your computer

- Git and Bun (already installed for this project).
- Node.js LTS, needed for the Supabase CLI via npx.
- For path B: the Firebase CLI. For path C: nothing extra, as Amplify builds in the cloud.

> Use one shared “owner” email (for example tech@yourchurch.org) for every service, with two-factor sign-in turned on. Never tie the service to one person’s private email.

PHASE 0 · AN AFTERNOON

Put the demo online (Vercel)

This makes Rhemlo reachable at a real address today, still on the demo server. It is perfect for testers and stakeholders.

1 Push your latest code to GitHub

```
git push origin master
```

2 Import the project in Vercel

- vercel.com → **Add New…** → **Project** → pick `rhemlo_app`.
- Vercel reads `vercel.json`: build `bun run build`, output `dist`, page rewrites and caching are already set.
- Leave environment variables empty: no variables means demo mode.

3 Press Deploy

In a few minutes you get an address like `rhemlo-app.vercel.app`. Every later push to master redeploys automatically; other branches get their own preview address.

PHASE 0 · CHECK IT

Check the public demo

- Open the address on a phone and a computer. Go through onboarding and one sitting.
- Sign in with a demo account (tiwa@demo.rhemlo.app, password shown on the sign-in sheet).
- Open a lesson in the Library and tap a reference: the KJV text should load.
- You → Downloads: download a quarter, switch the phone to airplane mode, reload. Lessons still open. This only works on the published site, not on your computer's dev server.
- You → Downloads → **Install Rhemlo** adds it to the home screen.

Vercel's free Hobby plan is meant for personal, non-commercial use. A church or ministry service with real users should check the terms and probably use Pro (about $20 per member a month at the time of writing).

Phase 0 done when testers can open the address and the demo works offline.

PHASE 1 · READ FIRST

What must be built before “live”

Switching to live is three settings. But the live database side is not finished yet. Be clear-eyed about these gaps before planning a launch date:

Gap	Why it matters
Migrations never run	The bundle’s six SQL files (0001–0006, 69 tables) have never been applied to a real database. Expect errors to fix on the first run.
Cross-device sync	In live mode, pulling progress back down is not implemented yet (it returns nothing). Needs a snapshot table and two functions.
About 35 newer features	Inbox, direct messages, rooms, coach mocks, consents, admin, verification and anomalies have no matching database functions yet, and the live adapter sends them generic arguments.
Email sign-in link	Live magic links return to `/auth/callback`, a page the app does not have yet.
Passkeys and push	Both are demo-only. Real passkeys need server checks; push needs keys and a sender.

> This is the next step of the journey: a focused build of these pieces, tested on a staging project, before any real learner signs in.

PHASE 1 · ALREADY THERE

What the migrations already give you

The bundle defines the core server logic. These database functions exist and match screens you have:

Area	Functions in the migrations
Progress	submit_reviews, refresh_lesson_progress, practice_set, grade_item
Leagues	join_league, leave_league, league_standings, my_league_position, create_league, league_preview, weekly_board, leaderboard_top
Daily challenge	daily_challenge, award_points
Matches	create_match, join_match, start_match, answer_match, match_reveal, match_state, finish_match
People	request_connection, respond_connection, friends_board, block_user, is_blocked
Chat + safety	my_chats, start_direct_chat, send_message, delete_message, can_direct_message, has_guardian_consent, report, moderate, moderation_queue
Circles + orgs	join_group, group_progress, group_daily, assignment_results, org_board, share_org, share_class
Manuals	my_editions, set_active_manual, retire_manual
Housekeeping	refresh_league_standings, ensure_log_partitions, prune_old_rows

Several newer screens can map onto these (for example DMs onto start_direct_chat and send_message). The rest need new functions.

PHASE 1 · STEP BY STEP

Create the Supabase projects

Make **two** projects: **rhemlo-staging** for testing and **rhemlo-prod** for real learners. Do everything on staging first.

1 supabase.com → New project

- Name, a strong database password (store it in a password manager), region near your learners.

2 Note three values

- Project URL: `https://<ref>.supabase.co`
- The **anon public** key: safe in the app.
- The **service_role** key: **secret**. Never put it in the app, in Vercel or in Git.

3 Turn on extensions

Database → Extensions: enable **pg_cron** (scheduled jobs). Enable any others the migrations name in a `create extension` line.

PHASE 1 · STEP BY STEP

Run the migrations

Copy the six SQL files from the bundle (`rhemlo/db/migrations`) into the repo so they are versioned:

```
npx supabase init
copy the 6 files into supabase/migrations/
npx supabase login
npx supabase link --project-ref <staging-ref>
npx supabase db push
```

The files run in number order. If one fails, read the error, fix the SQL in the repo, and push again. Commit every fix.

> Simpler alternative for a first look: Supabase dashboard → SQL Editor, paste each file in order (0001 to 0006) and run it.

> After it succeeds, open Database → Tables and check that **RLS enabled** shows on every table that holds learner data.

PHASE 1 · STEP BY STEP

Load the manual content

The app reads the manuals from static files, which is fast and works offline. The server also needs the question bank, so that ranked answers (daily challenge, leagues) are marked on the server and cannot be faked in the browser.

- Tables from `0001_content` and `0002_question_bank` hold editions, lessons and items.
- Write a small loader script that reads `public/data/<MANUAL>/` and inserts rows using the **service_role** key, run from your computer only.
- Record the content checksum and version so the steward screens show the real values.

> Never commit or upload the service_role key. Keep it in a local .env file that Git ignores.

PHASE 1 · STEP BY STEP

Build the missing pieces

Work through these as one project, on staging, in this order:

- **Snapshot sync**: a table for each learner's progress snapshot plus push and pull functions. Then fill in `pullSnapshot` in `src/lib/backend/live.ts`. The app already merges safely.
- **Auth callback page**: add the `/auth/callback` route that finishes an email-link sign-in.
- **Map the newer features**: replace the generic adapter at the end of `src/lib/backend/social.ts` with one call per feature, reusing existing functions where they fit.
- **Write the remaining functions**: inbox, rooms, coach mocks and submissions, consents, admin (manuals, seasons, organisations), sanctions, verification and anomalies. Each needs RLS rules.
- **Scheduled jobs**: with pg_cron, schedule the weekly league refresh and the clean-ups.
- **Decide on passkeys and push**: build them properly, or hide them in live mode for launch.

> Ask for this as its own piece of work: "Build the Rhemlo live backend gaps on Supabase staging." It is several focused sessions of work plus testing.

Sign-in settings

Supabase → Authentication:

- **URL configuration**: Site URL = your final address (for example https://rhemlo.app). Add redirect URLs for staging and for Vercel previews, such as `https://*-yourteam.vercel.app/**`.
- **Email provider**: on; require email confirmation.
- **Email templates**: rewrite the confirm and magic-link emails in Rhemlo's voice.
- **Rate limits**: keep the defaults or lower them.
- **Password rules**: at least 8 characters, matching the app.

> Supabase's built-in email is only for testing and is heavily rate-limited. Set up your own sender (Phase 2) before real users arrive.

PHASE 1 · CONNECT THE APP

Switch the app to live

In Vercel → Project → **Settings** → **Environment Variables**, add these three, for the **Preview** environment only at first:

```
VITE_RHEMLO_BACKEND = live
VITE_SUPABASE_URL = https://<staging-ref>.supabase.co
VITE_SUPABASE_ANON_KEY = <staging anon key>
```

- These are baked in when the app is built, so **redeploy** after changing them.
- Push a branch, for example `staging`: Vercel builds a preview on the live staging backend while production stays on the demo.
- The demo banner and the demo accounts disappear automatically in live mode.

> Only the anon key ever goes here. If you ever paste the service_role key into Vercel by mistake, rotate it in Supabase immediately.

PHASE 1 · PROVE IT

Staging test checklist

Use two phones and one computer, with fresh test accounts.

- Sign up, confirm the email, sign out, sign in again.
- Study on phone A; sign in on phone B and see the same progress.
- Go offline mid-sitting, finish, come back online: progress syncs, nothing is lost or doubled.
- Daily challenge: points show pending, then final; refreshing never gives a second attempt.
- League table, private league invite link, friendly match.
- Under-18 account: no messages until guardian consent; adult account: messages work, with the limit of 20 a minute.
- Report a message; act on it as a moderator; the user sees the notice in their inbox.
- Coach sets a mock; a learner submits; the coach sees the score.
- Admin retires a test manual; learners see it as read-only.

Phase 1 done when every line passes on staging, twice, with no console errors.

PATH B · GOOGLE

Host on Firebase Hosting

Use this instead of Vercel if you prefer Google. The backend stays on Supabase.

```
npm install -g firebase-tools
firebase login
firebase init hosting
  public directory: dist
  single-page app: Yes
  GitHub deploys: Yes (optional)
```

Then replace `firebase.json` with:

```
{ "hosting": {
  "public": "dist",
  "ignore": ["firebase.json", "**/.*"],
  "rewrites": [{ "source": "**",
                 "destination": "/index.html" }],
  "headers": [
   { "source": "/sw.js", "headers": [
     { "key": "Cache-Control", "value": "no-cache" }] },
   { "source": "/assets/**", "headers": [
     { "key": "Cache-Control",
       "value": "public, max-age=604800" }] }
  ] } }
```

Real files (assets, data) are served first; every other path falls back to the app. That is the same behaviour as vercel.json.

PATH B · GOOGLE

Build and deploy on Firebase

Firebase only serves files, so the three `VITE_` settings must be present when **you** build. Put them in a `.env.production` file (never commit secrets; the anon key is public but keep the habit):

```
bun install
bun run build
firebase deploy --only hosting
```

- Use a second Firebase site, or `firebase hosting:channel:deploy staging`, for a staging copy.
- Custom domain: Firebase console → Hosting → **Add custom domain**, then add the DNS records it shows. HTTPS is automatic.
- If you enabled GitHub deploys, add the VITE_ values as repository secrets and make the generated workflow run `bun install` and `bun run build`.

> Check Firebase's free (Spark) transfer limit against your traffic. With about 100 MB of artwork and manual data, many first-time visitors can use it up quickly; the Blaze plan is pay-as-you-go.

PATH C · AWS

Host on AWS Amplify

Amplify builds from GitHub and serves through CloudFront. The backend stays on Supabase.

- AWS console → **Amplify** → Create new app → GitHub → pick the repo and branch.
- Add the three VITE_ environment variables in the Amplify app settings.
- Use this build file (add it to the repo as `amplify.yml`):

```
version: 1
frontend:
  phases:
    preBuild:
      commands:
        - npm install -g bun
        - bun install
    build:
      commands:
        - bun run build
  artifacts:
    baseDirectory: dist
    files:
      - '**/*'
  cache:
    paths:
      - node_modules/**/*
```

AWS rewrites and domain

In Amplify → **Rewrites and redirects**, add one rule so app pages load on refresh while real files are served as files:

```
Source:
</^[^.]+$|\.(?!(css|gif|ico|jpg|js|png|txt|svg|
woff|woff2|ttf|map|json|webp|gz|webmanifest)$)
([^.]+$)/>
Target: /index.html
Type: 200 (Rewrite)
```

(Enter the source as one line with no breaks.)

- Custom domain: Amplify → **Domain management**. If your domain is in Route 53, it is configured for you; otherwise add the records it shows.
- For a staging copy, connect a second branch (for example staging) with staging values for the VITE_ settings.
- Advanced alternative: S3 bucket + CloudFront distribution with a 403/404 → /index.html error rule. More control, more setup.

PHASE 2 · POLISH

Your domain and HTTPS

1 Buy the name

For example rhemlo.app or rhemlo.org. Turn on auto-renew and registrar lock.

2 Point it at your host

- Vercel: Project → Settings → Domains → add it, then copy the records shown (usually an A record for the bare domain and a CNAME for www).
- Firebase or Amplify: the same idea in their consoles.

3 Wait for HTTPS

Certificates are issued automatically, often within an hour. Redirect www to the bare domain (or the reverse) so there is one address.

4 Update Supabase

Set the Site URL and redirect URLs to the final domain, or email links will point to the old address.

PHASE 2 · POLISH

Email that arrives

Sign-in links, guardian consent requests and moderation notices must reach inboxes, not spam.

- Create a Resend or Postmark account and add your domain.
- Add the **SPF**, **DKIM** and (recommended) **DMARC** DNS records they give you, and wait for “verified”.
- Supabase → Project Settings → Auth → **SMTP**: enter the provider’s host, port, user and password; sender such as no-reply@rhemlo.app.
- Send yourself a magic link and a password reset; check they land in the inbox on Gmail and Outlook.

> Use a sending subdomain such as mail.rhemlo.app so your main domain’s reputation stays clean.

PHASE 2 · MUST DO

Security and privacy checklist

- Every table with learner data has RLS on, and each rule is tested with two different accounts.
- Only the anon key is in the app; the service_role key lives only on your computer or a server.
- Two-factor sign-in on GitHub, Vercel/Firebase/AWS, Supabase, the registrar and the email provider.
- Supabase backups: on the paid plan, turn on daily backups (point-in-time recovery if budget allows) and practise one restore on staging.
- Children’s data: birth year only, guardian consent recorded, under-18 messaging rules on (already built).
- Publish a privacy policy and terms. For Nigerian users, follow the Nigeria Data Protection Act 2023; take local advice.
- Name a person who handles safety reports and data requests, with a response time.

> Do not invite real learners until every line here is ticked.

PHASE 2 · POLISH

Monitoring

What	Tool	Why
Front-end errors	Sentry (free tier)	See crashes on learners' phones, with the screen they were on
Uptime	Better Stack, UptimeRobot or similar	Get a message if the site or Supabase is down
Database health	Supabase reports and logs	Slow queries, connection count, storage
Traffic	Vercel Analytics, Firebase or Amplify metrics	Visitors and Web Vitals, privacy-friendly

Watch the first two weeks closely, especially Sunday evenings after class, when everyone studies at once.

BUDGET

Rough monthly costs

Indicative only, at the time of writing. Check each provider's pricing page.

Item	Start	Growing
Web host (Vercel Pro / Firebase Blaze / Amplify)	About $0–20	About $20–60, mostly bandwidth
Supabase	Free for staging; Pro about $25 for production	Higher compute add-ons as users grow
Domain	About $10–20 a year	Same
Email (Resend / Postmark)	Free tier to about $15	By volume
Sentry, uptime	Free tiers	About $0–30

Supabase's free projects pause after about a week without activity. Fine for staging, never for production.

The biggest bandwidth cost is first-time visitors downloading artwork and manual data. Compressed images and long cache times (already set) keep it down.

LAUNCH DAY

Go-live checklist

- Production Supabase project created, migrations run, content loaded, cron jobs scheduled.
- Production VITE_ settings added for the **Production** environment; redeployed.
- Your own account signs up, studies, syncs to a second device and appears in a league.
- Admin, coach and steward roles granted to the right people in the database.
- Email arrives; sign-in links open the right domain.
- Error and uptime alerts reach your phone.
- A short announcement to testers, then one parish or class, then everyone.

> Launch in waves: one class for a week, then a province, then everyone. Each wave shows problems while they are still small.

IF SOMETHING GOES WRONG

Rolling back

- **Bad release**: Vercel → Deployments → choose the last good one → **Promote to Production**. It takes seconds. Firebase and Amplify have the same “roll back to previous” buttons.
- **Backend trouble**: put the site in maintenance with a banner, or temporarily set `VITE_RHEMLO_BACKEND` back to demo and redeploy (learners then study on their own device only).
- **Data mistake**: restore the Supabase backup into staging first, check it, then restore production.
- **Leaked key**: rotate it in Supabase at once, update Vercel, redeploy.

> Write the names of who can do each of these, and keep this page where they can find it.

YOUR NEXT STEP

The journey from here

#	Step	Rough time
1	Phase 0: put the demo on Vercel for testers	An afternoon
2	Create Supabase staging; run migrations 0001–0006 and fix errors	1–3 days
3	Load content into the database	1 day
4	Build sync, auth callback, and map existing functions	1 week
5	Write the missing functions (inbox, rooms, mocks, consents, admin, verification)	1–2 weeks
6	Staging test checklist, twice	3–5 days
7	Domain, email, security, monitoring	2–3 days
8	Production project and launch in waves	1–2 weeks

> Start with step 1 today. It costs nothing and puts Rhemlo in testers' hands while the backend is built.

WORDS USED

Glossary

Term	Meaning
Static app	An app made only of files the browser downloads; no server code of its own
CDN	A worldwide network of servers that delivers files from close to each visitor
Supabase	A hosted Postgres database with sign-in and an automatic API
Migration	A versioned SQL file that creates or changes database tables
RLS	Row-level security: rules saying who may read or change each row
RPC	A database function the app calls by name
anon key	Public key for the app; safe only because of RLS
service_role key	Master key that ignores RLS; secret
Environment variable	A setting given to the build, such as VITE_SUPABASE_URL
Staging	A private copy for testing before production
pg_cron	Postgres add-on that runs jobs on a schedule
SPF / DKIM / DMARC	DNS records proving your emails are really from you